3X-UI Manager — User Manual

🇬🇧 English · 🇷🇺 Русский · 🇨🇳 中文

App version: 26.10.28.80. This manual is based on and current for that version.

Get the app: F-Droid · GitHub Releases · source code — see section 2 for the sources and the differences between builds.

> A full guide to **3X-UI Manager**, the Android app that manages a 3x-ui panel from your phone: dashboard, inbounds, clients, nodes and Xray settings.
>
> Names and labels match the app's interface. The English interface calls *inbounds* and *outbounds* **Inbounds** and **Outbounds**, and so does this manual.

Contents

- 1. What this is and what you need
  - 1.1. What the app does
  - 1.2. Requirements
  - 1.3. What the app does not do
- 2. Installing and updating
  - 2.1. Where to get it
  - 2.2. How the builds differ
  - 2.3. How updates arrive
  - 2.4. Update channel
- 3. Connecting to a panel
  - 3.1. Where to get the API token
  - 3.2. The connection form
  - 3.3. When the connection fails
- 4. Several panels
- 5. App lock
- 6. Dashboard
  - 6.1. Xray state
  - 6.2. Server metrics
  - 6.3. History charts
  - 6.4. Clients online
  - 6.5. Traffic this month
  - 6.6. Panel version and geo databases
- 7. Inbounds
  - 7.1. The list
  - 7.2. Editor: common fields
  - 7.3. Transport and security
  - 7.4. Sniffing, fallbacks and raw JSON
  - 7.5. Port reachability monitoring
  - 7.6. Protocols with special handling
- 8. Clients
  - 8.1. List, search and filters
  - 8.2. Creating and editing
  - 8.3. Handing a configuration to a client
  - 8.4. Bulk actions
  - 8.5. IP log
  - 8.6. Export, import and cleanup
  - 8.7. Groups
- 9. Nodes (multi-panel)
  - 9.1. List and status
  - 9.2. Adding a node
  - 9.3. Mutual TLS (mTLS)
  - 9.4. Updating the panel on a node

- 10. Xray: outbounds, routing, DNS
  - 10.1. Outbounds
  - 10.2. Testing outbounds
  - 10.3. Outbound subscriptions
  - 10.4. Routing
  - 10.5. Balancers and observatory
  - 10.6. Route test
  - 10.7. DNS
  - 10.8. General settings and logs
  - 10.9. Geodata auto-update
  - 10.10. The full Xray config
- 11. Panel administration
- 12. Backups
- 13. Alerts
  - 13.1. How it works
  - 13.2. Kinds of alert
  - 13.3. Configuration
- 14. App settings
- 15. About
- 16. Privacy and security
- 17. Panel version compatibility
- 18. Troubleshooting

1. What this is and what you need

1.1. What the app does

3X-UI Manager is a **management client** for the 3x-ui panel. It connects to a panel that is already running and gives you from your phone roughly what the web interface gives you:

- a dashboard with server and Xray state, load charts and the list of clients online;
- full work with **inbounds** — create, edit, enable/disable;
- work with **clients**: quotas, expiry, groups, search and filters, bulk actions, handing out links and QR codes, IP log;
- management of **nodes** (the master/node multi-panel setup), including mTLS;
- editing the **Xray** configuration: outbounds, routing, balancers, DNS, logs, and when needed the whole config at once;
- **administration** of the panel itself: account, API tokens, mail settings, restart;
- **backups** of the panel database;
- **local notifications** about problems: expired clients, exhausted quotas, an unreachable port, a stopped Xray, a node that dropped off.

1.2. Requirements

Item	Requirement
Android	7.0 or newer (API 24)
3x-ui panel	**v3.3.0 or newer** — that is the version from which the panel accepts API-token auth. Some capabilities arrived later and are unavailable on an old panel. Tested up to **v3.11.0**
Access	the phone must be able to reach the panel's address (directly, over a VPN, or through a reverse proxy)

Keeping the panel on its latest version is recommended — that way every section of the app is available, along with the panel's own fixes. What is tied to which version is listed in section 17. When the panel is older than a given feature needs, the app does not hide the section: it says inside it that your panel version doesn't support this yet and suggests updating.

1.3. What the app does not do

This is **not a VPN client**. It does not connect your phone to your proxies and does not route traffic through them — it only manages the panel. To use the connection itself you need an ordinary client (v2rayNG, Hiddify, Streisand and so on); the app helps you hand it a link or a QR code.

The app also does not install or set up the panel on a server — the panel has to be deployed already.

2. Installing and updating

2.1. Where to get it

Source	Link	Notes
F-Droid	https://f-droid.org/packages/net.yukh.xui	recommended: updates arrive through the catalog
GitHub Releases	https://github.com/yukh975/3X-UI-Manager/releases	download the APK by hand
Obtainium	add the repository `https://github.com/yukh975/3X-UI-Manager`	auto-updates straight from GitHub

A GitHub release carries **two** APKs. The ordinary one is named `3x-ui-manager-<version>.apk` — that is the one to take. The `fdroid.apk` file is the reference binary for F-Droid's build server; there is no reason to install it by hand.

How to read the version number. The version is the build date and a running build number, separated by dots: `26.10.28.80` is build 80 of 28 October 2026. The release tag is the same string with a `v` in front (`v26.10.28.80`). The build number only grows and is never reset, so installs with the old numbering (0.15.x) see the new version as an ordinary update.

2.2. How the builds differ

The app is built in two variants, and the only difference is the update mechanism:

- the **standard build** (GitHub, Obtainium) can check for updates itself and offer to install them;
- the **F-Droid build** cannot, because the catalog updates apps itself and F-Droid's rules forbid an app from downloading and installing its own APK. It doesn't even carry the corresponding system permission.

Both builds are signed with the same key, so you can install one over the other without uninstalling and without losing data.

2.3. How updates arrive

The standard build checks for a newer version at startup and shows a dialog with the list of changes. The text follows the interface language: with Russian selected you get the Russian changelog, not the English release body. If you have skipped several versions, the dialog shows the changes of **every** version after the installed one — newest first, each under a "Version … · date" heading.

You can check manually under **Settings** → **Check for updates** (the button sits below the update channel selector, see 2.4); it is no longer in About.

A dismissed dialog does not come back: the app remembers the version you declined and stays quiet about it on later launches. The version card on the dashboard remains as a reminder, and a manual check shows the update regardless.

The F-Droid build has no update check at all, and About says: "Installed from F-Droid — updates come through the F-Droid catalog."

2.4. Update channel

Standard build only. **Settings** → **Update channel**:

- **Stable** (the default) — tested public releases from GitHub;
- **Experimental** — pre-release builds as soon as they are made; they can be unstable.

The channel decides where the app looks for a new version, both at startup and on the "Check for updates" button.

---

3. Connecting to a panel

The app connects **with an API token**. The administrator's login and password are not used — a token is both safer and doesn't break when the password changes.

3.1. Where to get the API token

1. Open the panel in a browser.
2. Go to **Settings** → **Security** → **API Token**.
3. Create a token and copy its value.

> **A 3x-ui token with the `admin` scope is full administrator access.** Whoever holds it can do everything you can do through the web panel. Treat it like a password; if the phone is lost, delete the token in the panel and the app loses access.

3.2. The connection form

Field	What to enter
Name (optional)	a name for the panel, shown in the panel switcher instead of the address. Empty — the host is shown
Panel URL	the address including scheme and port, e.g. `https://panel.example.com:2053`. If your administrator set a secret path (`webBasePath`), include it
API token	the value copied from the panel. The "Show token" button lets you check that it pasted in full
Allow self-signed TLS	turns off certificate verification. Enable it only if the panel uses a self-signed certificate — and only for your own panel
Subscription base URL (optional)	rarely needed. The app reads the subscription address from the panel settings itself; set it here only when the public address differs — for example a panel behind a reverse proxy: `https://host:2096/sub/`

Once the connection succeeds the profile is saved, and the app reconnects on its own next time.

3.3. When the connection fails

- **An "unexpected response" error** usually means the app received HTML rather than JSON. That happens when `webBasePath` is missing from the address, or a reverse proxy answers with its own page.
- **A certificate error** — turn on "Allow self-signed TLS".
- **401, or "your API token is no longer valid"** — the token was deleted or disabled in the panel; create a new one.
- **Nothing answers** — check that the panel's address is reachable from the phone (mobile network, VPN, source-IP restrictions on the server).

---

4. Several panels

The app stores any number of panels and switches between them.

The ⇄ button in the top bar opens the list of saved connections ("Panels"), where you can:

- **switch** to another panel — every screen reloads against it;
- **add a panel** — an empty connection form opens;
- **edit** a saved panel — the panel's "Edit" button opens the "Edit panel" screen with the same fields as when connecting, the token already filled in;
- **sign out** of one panel — the app forgets its address and token.

When you edit a panel, the name and the subscription base URL are saved at once. If the panel URL, the token or the TLS setting changed, the app first checks the new details with a request to the panel and only then saves them. An address that another saved panel already has is not accepted: "Another saved panel already uses this URL".

In the list a panel is labelled with its name, or with its host when it has none. The active panel is named in the title bar as well, so mixing up servers is hard. Signing out of the last panel returns the app to its fresh-install state — which also clears the app-lock passcode.

---

5. App lock

Since the app holds tokens with full access to your servers, you can lock it behind a passcode. Under **Settings → App lock**:

- **Set passcode** — 4 to 8 digits;
- **Unlock with fingerprint** — fingerprint or face, if the device supports it;
- **Remove passcode**.

How it behaves in practice:

- at startup the passcode is asked for when one is set and a saved session was restored automatically; right after you typed a token in by hand, you are not asked again;
- sending the app to the background re-locks it, but with a **30-second grace period** — a quick switch to another app (to copy an address or read a code) and back doesn't prompt for the passcode;
- unlocking works with either the passcode or biometrics.

---

6. Dashboard

The first tab. It refreshes on its own; pull the list down to refresh at once.

6.1. Xray state

The card shows whether Xray is running, and its version. The buttons:

- **Restart** — briefly drops every active client connection, which the app warns you about. On panel 3.3.1 and newer it is usually not needed to apply configuration changes — see section 10;
- **Stop** / **Start**.

After you press one, the app holds the expected state for a moment so a lagging poll doesn't flip the button back and forth.

6.2. Server metrics

- **CPU**, **Memory**, **Disk** — as percentages;
- **Net ↑ / ↓ per s** — current rate;
- **Connections** — TCP and UDP counts;
- **Load 1·5·15m** — system load average.

Swap and server addresses. If the server has swap, a **Swap** tile appears next to CPU, memory and disk — tapping it opens its history, like the others; with no swap there is no tile. The server's public addresses are shown labelled **IPv4** and **IPv6** rather than as a single address.

6.3. History charts

The metric blocks are tappable and open a chart over a period. The interval is switchable (the default is the "live" two-minute bucket); the panel keeps system metrics history for roughly a week. The data comes from the panel — the app stores nothing of its own.

6.4. Clients online

The online counter is tappable and opens the list. If nodes are attached to the panel, the list is grouped by server, so you can see who is on the main server and who is on a node.

6.5. Traffic this month

Total traffic across all inbounds for the current period, again broken down by server. It is summed from the inbound counters; if some inbound is not on a monthly reset, the app flags that — otherwise the sum would mix monthly and all-time counters.

6.6. Panel version and geo databases

A separate card shows the panel version and offers an **update** when one is available. Next to it is the **geo database** update button (geoip / geosite): the panel re-downloads them and restarts Xray.

7. Inbounds

7.1. The list

Each inbound is a card: remark, protocol, port, traffic counters, expiry and the live ↑/↓ rate while the screen is open. The switch enables and disables the inbound; if the panel rejects the operation, the app puts the switch back and shows the error.

7.2. Editor: common fields

- **Remark** — the name you recognise it by;
- **Protocol** — on creation you choose from vless, vmess, trojan, shadowsocks, mixed (SOCKS and HTTP on one port; it replaced the former socks) and http, plus **TUIC** (panel 3.8.0) and **MASQUE** (panel 3.10.0) — for those, see 7.6. Inbounds of other protocols created in the web panel are shown and editable in the app;
- **Port** and **listen address**;
- **Enabled** — a disabled inbound is kept but does not run;
- **Traffic quota** and **expiry** for the whole inbound. The "Never" and "Pick date" buttons sit in a row below the date;
- **Traffic reset** — including the day of the month, if the panel supports it (3.6.0);
- **Disable XTLS flow** (VLESS only, needs panel 3.7.0) — stops the panel from injecting `xtls-rprx-vision` into this inbound even when its transport allows it. Vision keeps working on your other inbounds in the same subscription;
- **Node** — which server to deploy the inbound on, when nodes are attached.

For **Shadowsocks** you pick the **"Method"**: aes-256-gcm, chacha20-poly1305, chacha20-ietf-poly1305, xchacha20-ietf-poly1305, 2022-blake3-aes-128-gcm and 2022-blake3-aes-256-gcm (the default). For the 2022 methods the app generates a server key of the right length itself — you can "Regenerate" it, and it is recreated when the method changes. The 2022-blake3-chacha20 method is not in the list: the panel does not accept it for an inbound.

MTProto has a **"Max pending handshakes per IP"** field (panel 3.10.2, mtg-multi 1.16.0 and newer): new connections from an address that already has that many unfinished handshakes are closed; `0` turns it off.

Saving asks for confirmation — "Create inbound?" or "Apply changes to this inbound?" — and leaves alone the fields the form doesn't have: the tag, the order in the subscription, the exclusion from the subscription and the traffic-reset day stay as they were.

7.3. Transport and security

Transport
(`streamSettings`) is edited through structured fields: the transport type (TCP/raw, WebSocket, gRPC, HTTPUpgrade, XHTTP, KCP and so on), its parameters, and TLS or REALITY with their fields — certificates ("Certificate file", "Key file"), SNI, fingerprint, short ID. WebSocket with TLS gets ALPN `http/1.1` by default.

The editor doesn't offer transport and security options that would break the inbound: for MASQUE, XDRIVE and TUN, for example, the extra options are hidden. XDRIVE's storage is set in the transport settings JSON, and its security stays None.

An important detail: the app edits the transport as a live JSON object, so **keys it doesn't know about are not lost**. Editing an inbound that was created in the web panel with exotic settings will not wipe them.

7.4. Sniffing, fallbacks and raw JSON

- **Sniffing** — destination detection from traffic, with its list of types;
- **Fallbacks** — forwarding by SNI/path to other ports;
- **Advanced: protocol settings (JSON)** — protocol settings that aren't surfaced as form fields are available as raw JSON. Clients are kept separately and are not overwritten: they are managed on the Clients tab.

7.5. Port reachability monitoring

The editor of a saved inbound has a **"Monitor reachability"** switch: alerts will tell you if this inbound's port stops answering. It belongs to alerts and is meant for inbounds exposed directly, without a reverse proxy. The port and remark are remembered **on the phone itself**, and the background probe then knocks on that port directly without asking the panel. It works that way because the panel is often firewalled off from the phone, which makes its reachability a poor health signal.

7.6. Protocols with special handling

MASQUE (panel 3.10.0):

- the **"Address pool"** fields — the prefixes tunnel addresses are handed out from, at most one IPv4 and one IPv6; **MTU** (1280 to 65535) and the path;
- TLS is mandatory;
- clients sign in with email and password, so a MASQUE client's email cannot contain `:`;
- **there is no MASQUE link** — clients get the connection only from the JSON subscription (see 8.3).

TUIC (panel 3.8.0) is served not by Xray but by a separate tuic-server process. So a TUIC inbound has no transport, TLS/REALITY or sniffing sections: the certificate, SNI and QUIC options are set in the protocol settings JSON. The panel cannot enforce a single client's traffic limit or IP count on TUIC — the traffic limit is set on the inbound itself.

---

8. Clients

8.1. List, search and filters

The list shows, for each client: an online dot, email/name, ↑/↓ traffic against the quota, live rate, expiry and when they were last seen.

Search covers more than the email — also the comment, sub ID, UUID, password, the auth field and the Telegram ID.

Filters (the button with a count badge) mirror the web panel's:

- by status — active, disabled, expired, quota depleted;
- by group.

Within one section the conditions are OR'ed, and the sections narrow the selection independently — the same as in the panel.

8.2. Creating and editing

The main fields: **email/name**, **traffic quota** (in GB), **expiry**, **IP limit**, **group**, **comment**, **Telegram ID**, **subId**, and the binding to one or more inbounds. If the panel rejects binding or unbinding an inbound, the app shows the error.

The **group** is picked from the panel's list of groups — it includes empty groups that nobody is in yet. You can also type a new name: the panel will show that group in its list. The groups themselves are managed on a separate screen — see section 8.7.

The form adapts to the protocol of the selected inbound:

- for **MTProto** a FakeTLS secret (with a regenerate button) and an ad-tag appear;
- for **WireGuard** — the peer's allowed IPs;
- for **AmneziaWG** — **forwarded ports**: the ports and ranges the panel routes to this client, e.g. `80, 443, 8000–8100`;
- for **WireGuard and AmneziaWG** — **"Keepalive (seconds)"** (panel 3.8.0): how often the client sends a keepalive. `25` (the value for a new client) keeps NAT open and brings an idle peer back after a drop; `0` turns it off. An **AmneziaWG** client on panel 3.11.0 can also be given a range, e.g. `15–25`: 0 to 65535, the lower bound first. If the client is also bound to a WireGuard inbound, a range is not accepted — WireGuard takes whole seconds only;
- for **TUIC** — a warning that a single client's traffic and IP limits don't work on a TUIC inbound (see 7.6);
- for VLESS — the **flow** field.

Renewal and traffic reset (needs panel 3.7.0). Beyond the familiar "auto-renew every N days" there are now:

- **renew on a day of the month** — the client renews on that date every month instead of on a fixed interval; `0` keeps the old mode. A month too short for the chosen day renews on its last day;
- **max renewals** — how many times auto-renew may fire before the client is left to expire; `0` means no limit;

- **the client's own traffic-reset cycle** (never / hourly / daily / weekly / monthly), with a day of the month for the monthly one. The reset period used to be an inbound-level setting only.

If a client has weekly renewal set in the web panel, the app says so and keeps it on an ordinary edit; only a reset period or a renewal day chosen in the form turns it off. Likewise, an edit doesn't lose the VLESS reverse proxy (reverse).

Device limit and the device list (needs panel 3.7.0). The **device limit** caps how many devices may fetch this client's subscription; `0` means unlimited. The **"Devices"** button (on a saved client) lists the registered devices: model and OS, when each was last seen, and on panel 3.8.0 also the **"Device fingerprint"**. A device can be removed one at a time or the list cleared entirely: this is how you unlink a phone, since a removed device simply registers again on its next subscription fetch.

Identifiers (UUID, password, subId) are minted by the app on creation in the same format the web panel uses.

Expiry down to the minute. After the date, the app asks for a time of day; a client without an expiry gets 23:59. The time is entered in the phone's time zone — as in the panel's own web form — and that zone is labelled under the field. If the panel is set to a zone of its own, the same moment on the panel's clock is shown next to it, so a panel abroad and an admin at home don't have to guess whose midnight it is.

8.3. Handing a configuration to a client

Tapping a client opens the "Share" sheet:

- **Subscription** — a single link from which a client app fetches every configuration and keeps them updated. A QR code is shown, with copy and "Share" buttons;
- **JSON subscription** — the Xray configurations as JSON; shown if the panel serves it. MASQUE connections are **only in this one**;
- **Encrypted Happ link** (panel 3.8.0) — the "Create encrypted link" button gets a `happ://crypt5/…` link from the panel, with a QR code; you can "Regenerate" it. Only the Happ app opens it, but anyone who has it may still recover or share the subscription URL. If the feature is turned off, the app tells you where to enable it in the panel: **Subscription** → **Happ** → **Subscription links**, "Encrypted subscription links";
- **Connections** — the individual per-server links; each expands to its own QR;
- **"As the subscriber sees it"** — the live subscription status (online, used against the quota) exactly as the client's own app receives it. Requires panel 3.6.0; if the subscription server isn't reachable from the phone, the block is simply omitted.

A link too long for a QR code is shown without one — you can copy or share it. The same goes for links with post-quantum keys: their QR would come out too dense.

Edit, **Delete**, **IP log** and **"Reset traffic"** are available from here as well: the used traffic starts from zero, and a client that its quota switched off is enabled again. The reset asks for confirmation.

8.4. Bulk actions

A long press turns on selection mode. With the selected clients you can:

- **enable** or **disable** them;

- **adjust** them (the "Adjust" button) — add or subtract days and gigabytes, set the flow. On panel 3.8.0 the same place sets the **"Device limit"** (empty keeps each client's limit; lowering it removes the extra registered devices) and the **"MTProto ad tag"** (32 hex characters to set, `none` to clear, empty to keep; MTProto inbounds only). After the operation the app reports how many clients it changed and why it skipped any;
- **reset traffic** — with confirmation;
- **delete** them.

8.5. IP log

Shows which addresses the client connected from and when, and with multi-panel, through which node. The log can be cleared.

An empty log is not a fault: the panel keeps an address for only **30 minutes** after the client's last connection, so a client that hasn't connected lately has an empty log — and the app says so.

8.6. Export, import and cleanup

- **Export** all clients as JSON — useful as a backup or for moving them;
- **Import** from the same JSON; the panel skips clients whose email already exists;
- **Delete unbound clients** — remove clients not attached to any inbound.

8.7. Groups

The **"Groups"** item in the ⋮ menu on the Clients screen opens the panel's list of groups — including empty ones that nobody is in yet. Each group shows its client count and traffic: sent, received and total.

Tapping a group expands the list of its clients with ↑/↓ traffic and a "disabled" mark; tapping a client opens its sheet. An empty group says "This group has no clients yet."

The **"Create group"** button adds a new group; the other actions are in the ⋮ menu of the group itself:

- **Add clients…** and **Remove clients…** — pick clients from a searchable list. A client belongs to one group only, so adding it moves it out of its previous group; its inbound bindings don't change. Removed clients stay in the panel — just without a group;
- **Rename** — the new name goes to the group and to all of its clients;
- **Reset traffic** — zeroes only the group's counter; the clients' own traffic and limits don't change. Needs panel 3.4.2;
- **Delete group** — the group disappears, its clients stay in the panel without a group;
- **Delete the group's clients** — deletes all of the group's clients from the panel along with their traffic. This cannot be undone, so the action asks for a separate confirmation.

A group's traffic counts from its last reset, so it may be less than the sum of its clients' traffic. On panel 3.3.0 a group shows only total traffic — the split into sent and received arrived in 3.3.1.

---

9. Nodes (multi-panel)

This section matters if you run a master panel with nodes: several servers you manage from one place.

9.1. List and status

Each node shows its name, address, state, the panel version running on it and counters. The switch enables and disables the node. Pull the list down to refresh.

9.2. Adding a node

The fields: name, scheme (http/https), address, port, base path, the **node's API token**, **"TLS verify"** and **"Connection outbound (empty = direct)"** — the outbound for the connection to the node, picked from the outbound tags of the Xray config.

"TLS verify" takes the values `verify` (normal verification), `skip` (no verification), `pin` and `mtls`. In `pin` mode a **"Certificate SHA-256 pin"** field appears — the SHA-256 of the node's certificate in hex or base64.

Since panel 3.6.0 the node token is **not returned back** — the panel only reports whether one is set. So when editing, an empty token field means "keep the current one", and entering a value replaces it. Editing a node doesn't reset the sync of the selected inbounds.

9.3. Mutual TLS (mTLS)

A separate **"Node mTLS"** page in the ⋮ menu (panel 3.4.0 and newer). It covers two things:

- **copy this panel's CA**, to register it as trusted on a node;
- **set the CA** whose client certificates this panel trusts when it acts as a node.

Since panel 3.7.0 it also has **"Reload master credential"**: an updated master certificate is applied in place, without restarting the panel — a rotation used to take effect only after a restart.

With mTLS a node needs no API token.

9.4. Updating the panel on a node

A node's panel can be updated straight from the app. The node downloads the new build and restarts, which takes noticeably longer than a single list refresh — so the app keeps polling the node until its reported version actually changes, and only then shows the result.

---

10. Xray: outbounds, routing, DNS

Everything to do with the Xray configuration sits in the ⋮ menu in the top bar: **Outbounds**, **Routing**, **Balancers**, **DNS**, **General / Logs**, **Geodata auto-update** and **Xray config**. The sections edit different parts of one and the same config, and each of them saves the config as a whole — the neighbouring parts you didn't touch are preserved.

The outbound, rule, balancer and DNS server editors have a **"Save"** button — the edit goes to the panel at once. The system Back button returns to the list. If the tag is empty or already taken, the form scrolls to the error.

When changes take effect. Panel 3.3.1 and newer applies a saved configuration itself: on the fly where possible, restarting Xray only when there is no other way. The app says "Saved and applied", and nothing needs restarting by hand. The exception is editing an outbound subscription: after it the app says "Saved — restart Xray to apply" (refreshing, moving and deleting subscriptions the panel applies itself within 30 seconds). On panel 3.3.0 changes take effect **after an Xray restart** — the button is on the dashboard.

10.1. Outbounds

A list of outbounds with their tag, protocol and address. The first one is the default route and is labelled as such. Available actions:

- **create and edit** an outbound: protocol, address, port, credentials, transport, plus **Target Strategy** (from `AsIs` to `ForceIPv4`) and "Send through". The protocols include **AmneziaWG** (panel 3.8.0); for **Hysteria** and **MASQUE**, including WARP over MASQUE, the transport and TLS settings are available. TLS has a **"Use system CA"** switch (panel 3.10.0): the server is verified against the CA store of the server's operating system rather than Xray's built-in roots — needed for a private CA;
- **reorder** — with the up/down arrows; the order decides which one becomes the default route;
- **delete**;
- **import from a `vless://` link**;
- **import and export** the whole list as JSON.

10.2. Testing outbounds

Every outbound has a **"Test"** button with a mode selector:

Mode	What it measures
TCP	reachability and delay at the connection level
HTTP	a full request through the proxy; also reports the egress IP and country
Real delay	the full time including tunnel establishment — closest to the real experience

For HTTP and "Real delay", the result is joined by the IP, the country flag and a **WARP** marker when the egress goes through it. The URL used for the probe is set on the panel side.

10.3. Outbound subscriptions

The panel can fetch someone else's server list from a subscription URL itself and merge it into the config as outbounds. The section opens from the ⋮ menu on the Outbounds screen.

Each subscription is configured with:

Field	Meaning
Subscription URL	where the panel takes the list from
Remark	how you recognise it
Tag prefix	e.g. `hk–`, to tell its servers apart in routing
Update interval	how often the panel re-reads the URL
Enabled	switch it off temporarily without deleting
Before manual outbounds	put its servers above yours — then one of them can become the default route
Allow private address	permit a URL pointing at a local/LAN address; off by default for safety
Allow insecure TLS	for a source with a self-signed certificate

Actions: **preview** (the panel fetches and parses the URL, showing how many servers were found and with which tags — before you save), **refresh now**, **refresh all**, reorder and delete.

Servers that came from a subscription appear on the Outbounds screen as a separate "From outbound subscriptions" block. They can be **tested** and used in routing rules and balancers just like ordinary ones, but they **cannot be edited directly**: the panel regenerates them on every subscription refresh, so any edit would be overwritten anyway. Change them through the subscription itself.

10.4. Routing

The **Routing** section holds:

- **"Routing rules"** — by domain, IP, port, protocol, inbound; each rule points at an outbound or a balancer to send the traffic to. Rules can be enabled and disabled, reordered, imported and exported. A rule's comment is visible in the list, and on panel 3.8.0 it can be edited too — the "Comment (panel only)" field, up to 200 characters;
- **"Routing strategy"** — `AsIs`, `IPIfNonMatch` or `IPOnDemand`;
- the **"Test route"** button — see 10.6.

Traffic that matched no rule goes to the first outbound in the list — see 10.1.

The internal stats rule (`api`) is protected from being disabled — without it traffic accounting breaks.

If a routing script is set in General / Logs (see 10.8), the screen warns that Xray ignores these rules.

10.5. Balancers and observatory

Balancers have their own menu item, ⋮ → **Balancers**. They are groups of outbounds among which Xray picks according to a strategy. The balancer editor:

- **Tag** — what routing rules refer to;
- **Strategy** — `random`, `roundRobin`, `leastLoad` or `leastPing`;
- **Selector (comma-separated)** — tags or tag prefixes of the outbounds in the group;
- **Fallback** — the outbound to use when none in the group will do.

The `leastLoad` and `leastPing` strategies, as well as `random` and `roundRobin` with a fallback, need an **observer** (observatory) that probes the outbounds. The app creates and removes it itself — without an observer Xray failed to start with "not all dependencies are resolved". The **"Observatory"** block (panel 3.10.0) sets how it probes: "Probe URL" (must answer HTTP 204), "Probe Interval", "Probe Timeout", "Sampling Count", "HTTP Method" and other parameters. The list of watched outbounds follows the balancer selectors. When there is no observer, the screen says so: "No connection observer is active".

The screen is saved with its own **"Save"** button — it is active while there are unsaved changes. Changing the strategy doesn't reset the `leastLoad` settings.

10.6. Route test

This tool answers "where would this address go". Enter a domain or IP, optionally a port and network type, **pick an inbound** — and you get the resulting outbound, the balancer chain, or a "default outbound" verdict.

Choosing the inbound is essential rather than optional: nearly all routing rules key on the inbound, and without it the router cannot decide. The test works for local inbounds; the master panel has no routing rules for node inbounds, so the result isn't meaningful for them.

10.7. DNS

An editor for the DNS section: servers (plain and conditional), the query strategy, and static hosts entries.

Since panel 3.10.0 DNS can use a **"Lua Script"** — the path to a `.lua` file with a `HandleDNSQuery` function: the script answers every query and contacts the servers itself. For this a server has an **"ID"** field — the name by which the script picks the server. If the file is missing or has an error, Xray will not start.

Saving a server keeps its ID, tag, Client IP, expected IPs, "Skip fallback" and timeouts.

10.8. General settings and logs

The **General / Logs** section: log level, log access parameters, statistics and the other general core settings.

This is also where (panel 3.10.0) the **"Routing Script"** lives: the path to a `.lua` file with a `HandleRoute` function. The script then picks the route for every connection, and Xray ignores **all** routing rules, the panel's `api` rule included. Send the `api` inbound to the `api` outbound in the script itself, or traffic stats and live client changes stop. If the file is missing or has an error, Xray will not start.

10.9. Geodata auto-update

The geo-database button on the dashboard (see 6.6) fetches the files **now, by hand**. The **"Geodata auto-update"** section configures the same thing **on a schedule**: Xray downloads the listed files itself and reloads them without restarting. Needs panel 3.7.0.

- **Schedule** — a five-field cron expression, e.g. `0 4 * * *` (daily at 4 am).
- **Download through outbound** — optional; the list comes from the outbound tags in the config. Empty means the normal route.
- **Files** — the file name and where to fetch it from. The URL must be **HTTPS** and the name a plain one like `geosite_custom.dat`, with no path.

The app checks those three constraints before saving and will not let an invalid set through. Removing the last file drops the schedule block from the config entirely — a schedule with no files means nothing.

One constraint comes from Xray itself: **the file has to exist in the panel's `bin` folder once**, or there is nothing to update — so it gets there first via the dashboard button or by hand, and the schedule keeps it current from then on. Routing rules reference such files as `ext:geosite_custom.dat:category`.

10.10. The full Xray config

If a field you need isn't in any form, the configuration can be edited as text — the whole thing, the same JSON as on the panel's Xray Configuration page. The outbound test URL is set next to it. Saving asks for confirmation: the config in the panel is replaced and applied at once. Panel 3.3.1 and newer applies it itself — on the fly where possible; on 3.3.0 an Xray restart is needed.

---

11. Panel administration

The **Panel admin** section (the ⋮ menu) gathers the panel's own settings.

- **Admin account** — change the login and password. The current ones must be entered. After the change your API token keeps working, but other browser sessions are signed out.
- **API tokens** — the panel's token list: enable, disable, delete, create. A new token's value is shown **once** — copy it right away. Deleting a token immediately cuts off every app using it. Since panel 3.7.0 you choose the token's **scope** when creating it:
  - **Admin (full access)**;
  - **Monitor (read-only)** — server state and metrics history;
  - **Node sync** — what a central panel needs: inbounds, clients, Xray restart;
  - **Node administrator** — node sync plus starting a panel update; needs panel 3.11.0.

  You can also set an optional **lifetime in days**. In the list the scope and the expiry date show under the token's name, and an expired token is flagged. The app itself needs an `admin` token: with other scopes some sections won't work. Tokens issued earlier have the `admin` scope — their behaviour doesn't change.
- **IP limit allowlist** (needs panel 3.7.0) — comma-separated addresses and subnets (IP or CIDR) that the IP limit never counts and never bans. It is needed where many people sit behind one address: a shared office egress or a CGNAT carrier would otherwise use up a client's limit and get it blocked.
- **Subscription** — the **announcement** text (`subAnnounce`) shown as a banner on the subscription info page.
- **Email (SMTP)** — the sender address and name for the panel's mail. Leaving the address empty falls back to the SMTP username. This matters when the relay's login is not an email address: without a valid `From` , strict receivers such as Gmail reject the message. Requires panel 3.6.0.
- **Notifications** — the **"Outbound-down threshold"**: how many consecutive failed probes must happen before the panel sends an alert about an unreachable outbound. `1` means the old behaviour, alerting on the first failure. It helps against a flood of false alarms on a flaky link. Requires panel 3.6.0.
- **Restart panel** — restarts the panel service itself; the app's connection drops for a few seconds.

12. Backups

The **Backup / restore** section.

- **Back up to file** — the app fetches the panel database and asks where to save the file. The panel picks the name: `x-ui.db` for SQLite or `x-ui.dump` for PostgreSQL.
- **Restore from file** — pick a file; the panel imports it under its own engine. On panel 3.8.0 and newer the panel restarts itself after the import; the app waits until it answers again and reports "Restored — the panel restarted and is back". If the panel is still silent after a minute, the app says so. On older panels Xray is restarted after the import.

That database holds **everything**: panel settings, inbounds, clients and the Xray configuration. That makes a backup the quickest way to move to another server — and, for the same reason, a file to treat as a secret.

API tokens come from the backup too: if it doesn't contain this app's token, you will have to connect again after restoring. The app warns about this in the confirmation.

---

13. Alerts

13.1. How it works

Alerts are **entirely local**. The app itself polls the saved panels roughly every 30 minutes and raises system notifications. There is no external push service involved: neither your data nor your panel addresses go anywhere. The flip side is that an alert appears at the next check rather than the same second.

Reachability is checked **at the TCP port level**, not through the panel API. The reason is simple: the panel is often firewalled off from the phone, so its being unreachable says nothing about the health of the service. By default **port 443** is probed — the usual public entry point.

To avoid crying wolf, an unreachability alert fires only after **two consecutive failed checks**, and an authentication error (401) is not treated as unreachability.

13.2. Kinds of alert

Alert	When
Client expired / expires soon	the expiry has passed, or is closer than the configured number of days
Traffic limit almost reached	more than the configured percentage has been used
Panel unreachable	the configured port doesn't answer
Inbound unreachable / Inbounds unreachable	the port of an inbound marked for monitoring doesn't answer
Xray is down	the core isn't running
Node offline	a node is off the air

Notifications are split across **two system channels** — client ones and infrastructure ones. That lets you mute the client chatter in Android settings while keeping server alarms. Each notification carries the date and time it fired, so if you see it hours later you can tell whether the problem is current.

13.3. Configuration

Under **Settings** → **Panel alerts**:

- the **enable** switch — client expiry, traffic limits, inbound and node reachability (on Android 13+ the app asks for notification permission);
- **days before expiry** — how far ahead to warn (default 3);
- **traffic threshold (%)** — at what usage to warn (default 90);
- **panel port** — which port to probe for reachability (default 443).

Individual inbounds are added to monitoring from their editor — see 7.5.

14. App settings

- **Language** — **System default** (the default), English, Русский or 中文 (Simplified Chinese). In system mode the app follows the phone's language and switches as soon as you change it, without a restart. An explicit choice always wins over the system one.
- **Speed units** — bytes (KB/s) or bits (Kbit/s). Applies to inbound and client rates.
- **Update channel** and the **"Check for updates"** button — standard build only, see 2.4.
- **App lock** — see section 5.
- **Panel alerts** — see 13.3.

15. About

A separate item in the ⋮ menu. It holds:

- the name, **version** and copyright;
- in the F-Droid build, a note that updates come through the catalog. In the standard build the update check has moved to Settings, see 2.3;
- the **changelog** — the full list of versions and what changed in each. It works offline: the changelog is bundled into the app, nothing is fetched for it;
- a link to the **project on GitHub**;
- the **Our projects** block — the 3x-ui manual, Currency Converter and netadm.pro.

Before you connect to a panel the section is reachable from Settings, so the version can be checked without signing in.

16. Privacy and security

Where the app connects. Only to the addresses you entered yourself: your panel and, when handing a subscription to a client, its subscription server. Plus, in the standard build, the app's releases page for the selected update channel, to check for updates. There is no analytics, no advertising and no trackers in the app.

The F-Droid build never contacts our infrastructure at all — it doesn't even have the update check. A side effect of being honest about it: we have no idea how many people use the app from the catalog.

How your data is stored. Connection profiles (the panel address and API token) live in Android's encrypted storage: values are encrypted with AES-256-GCM, keys with AES-256-SIV, and the master key sits in the Android Keystore. On top of that, entry to the app can be locked with a passcode and biometrics.

Permissions the app requests:

Permission	What for
Internet and network state	talking to the panel
Notifications	local alerts
Biometrics	unlocking by fingerprint/face
Install packages	standard build only — to install an update; the F-Droid build doesn't have it

Worth remembering. A 3x-ui API token with the `admin` scope is full administrator access. The "Allow self-signed TLS" mode turns off certificate verification and is appropriate only for your own panel. An encrypted Happ link hides the subscription URL only from casual eyes: whoever has it can recover the URL.

---

17. Panel version compatibility

The minimum panel is 3.3.0: that is where the panel starts accepting the API-token auth the whole app is built on. It will work against that version, but some capabilities arrived in later panel releases. The app is tested with panels **up to and including 3.11.0**.

Capability	Panel needed
API-token sign-in, dashboard, inbounds, clients and their groups, subscriptions, Xray config	3.3.0
Outbound subscriptions, group traffic split into sent and received, Xray changes applied on save without a manual restart	3.3.1
Nodes and mTLS, client IP log, client export and import, live inbound speed, online split by server	3.4.0
Bulk client actions, VLESS encryption key generation	3.4.1
Client group traffic reset	3.4.2
Live per-client speed, outbound test, Target Strategy, route test, subscription announcement, MTProto and WireGuard client fields	3.5.0
SMTP sender settings, outbound-down threshold, "as the subscriber sees it" status, write-only node tokens, day-of-month traffic reset	3.6.0
Renewal on a day of the month, max renewals, the client's own traffic-reset cycle, the device limit and device list (HWID), AmneziaWG forwarded ports, API token scope and expiry, the IP limit allowlist, "Disable XTLS flow", reloading the master mTLS credential, geodata auto-update	3.7.0
WireGuard and AmneziaWG client keepalive, encrypted Happ link, bulk setting of the device limit and the MTProto ad tag, device fingerprint, AmneziaWG outbound, editing routing rule comments, TUIC inbounds, waiting for the panel restart after a backup restore	3.8.0
MASQUE inbounds and the JSON subscription, "Use system CA", Lua scripts for DNS and routing, DNS server ID, observatory settings, creating Shadowsocks inbounds with 2022 keys	3.10.0
"Max pending handshakes per IP" for MTProto (needs mtg-multi 1.16.0)	3.10.2
Keepalive range for AmneziaWG clients, the "Node administrator" token scope	3.11.0

Panel 3.9.0 added no new features to the app — everything that works with 3.8.0 works with it.

The recommendation is simple: run the latest panel. Then every section of the app is available, and you get the panel's own fixes along the way.

If the panel is older than a section needs, the app **does not hide it** — it shows an explanation instead: your panel version doesn't support this, update the panel. That way you can see the feature exists and know what to do about it.

18. Troubleshooting

"Unexpected response" or a parsing error. The app received something that isn't JSON. Most often the panel address is missing its secret path (`webBasePath`), or a reverse proxy intercepts the request. Check the whole address by copying it from the browser.

A certificate error. Turn on "Allow self-signed TLS" in the connection settings — but only for your own panel.

"Your API token is no longer valid". The token was deleted or disabled in the panel. Create a new one and connect again; the saved profile fills in the rest.

A section says the panel doesn't support the feature. The panel is older than required — see section 17. Update it from the dashboard.

Xray changes have no effect. On panel 3.3.1 and newer a save is applied at once. If after saving the app says "Saved — restart Xray to apply" (this happens after editing an outbound subscription), or the panel is older than 3.3.1, restart Xray with the button on the dashboard.

Xray won't start after setting up a balancer or a script. Check that the `.lua` file named in DNS or in General / Logs exists and has no errors — otherwise Xray won't start. `leastLoad` and `leastPing` balancers (and `random` and `roundRobin` with a fallback) need an observer — the app adds it itself, but if the config was edited by hand, check the "Observatory" block, see 10.5.

No alerts arrive. Check that they are enabled in settings, that Android granted the notification permission, and that battery optimisation isn't killing the app. Remember that the check runs roughly every 30 minutes, and unreachability is reported after two consecutive failed checks.

The route test always says "default outbound". No inbound was picked — rules almost always key on it. For node inbounds the result isn't meaningful: the master panel has no routing rules for them.

No app update arrives. In the F-Droid build updates come only through the catalog and appear a day or two after a release — that is their build server's schedule, and it is out of our hands. In the standard build, check the selected update channel and press "Check for updates".

Created from an analysis of the app's source code. Yuriy Khachaturian (yukh.net)

Licensed under CC BY 4.0.